

&j VISION — Architecture, Features & Design Documentation

Vision is a privacy-first, ephemeral, real-time communication platform featuring automatic IP subnet room matching, WebRTC P2P audio/video calling, host approval knocking systems, and a Uiverse dark-glass cyber-neon design.

Ø=ÜD 1. Architecture & Repository Structure

Vision is built with a decoupled Node.js + Express + Socket.IO backend and a Vite + React + TailwindCSS frontend.

```
``text
Vision/
% % % client/                                # Frontend Application (Vite + React)
%   % % % public/                            # Static assets & favicon
%   % % % src/
%   %   % % components/                     # UI Components & Modals
%   %   %   % % ApprovalModal.jsx           # Host Knocking & Candidate Pending Dialog
%   %   %   % % AuthModal.jsx               # Guest & Password Sign-in Panel (Uiverse)
%   %   %   % % BetaNoticeModal.jsx         # Feature status overlay
%   %   %   % % ChatArea.jsx                # Main chat stream, voice & file render
%   %   %   % % Header.jsx                  # Top bar, room pill, online count & controls
%   %   %   % % InitialLoader.jsx           # Telemetry startup loader
%   %   %   % % MessageInput.jsx            # Touch input, voice recorder, file upload
%   %   %   % % MobileNavBar.jsx            # Responsive navigation drawer for mobile
%   %   %   % % PasswordModal.jsx           # Touch PIN keypad prompt for protected rooms
%   %   %   % % ProfileSetupModal.jsx       # Avatar & handle customization grid
%   %   %   % % RoomCallModal.jsx           # Active P2P Audio/Video overlay
%   %   %   % % RoomModal.jsx               # Create/Join Room modal (Uiverse)
%   %   %   % % RoomSettingsModal.jsx       # Host controls & feature toggles
%   %   %   % % SettingsModal.jsx           # User profile & sound settings modal
%   %   %   % % ShareModal.jsx              # Compact Share Room & Live QR canvas modal
%   %   %   % % Sidebar.jsx                 # Online members drawer & room metadata
%   %   %   % % VideoCallModal.jsx          # WebRTC video grid container
%   %   % % % context/
%   %   %   % % SocketContext.jsx            # Socket.IO state, room actions, auto-purge
%   %   %   % % WebRTCContext.jsx           # Peer-to-peer WebRTC mesh & signaling engine
%   %   % % % styles/
%   %   %   % % index.css                   # Base CSS, Uiverse tokens, mobile dvh rules
%   %   % % % utils/
%   %   %   % % avatar.js                   # SVG seed avatar generator
%   %   %   % % sound.js                    # Web Audio API terminal sound effects
%   %   %   % % App.jsx                     # Root application container & modal router
%   %   %   % % main.jsx                    # React DOM entry point
%   % % % package.json
%   % % % vite.config.js                    # Vite build & proxy settings
%
% % % server/                               # Backend Telemetry & Signaling (Node.js)
% % %   index.js                            # Express server, Socket.IO setup, Rate limiting
% % %   ipUtils.js                          # IP extraction, /24 subnet grouping, IP hashing
% % %   roomManager.js                      # Room state store, message TTL purge, Host approvals
% % %   socketHandler.js                    # Socket.IO event router (rooms, chat, WebRTC, knocking)
% % %   routes/
% % %     api.js                            # REST API (/api/ip, /api/health, /api/room/:id)
% % %     upload.js                          # Encrypted media file upload endpoint
% % %     test_verification.js               # Core backend test script
% % %     test_full_suite.js                # 28-point full automated test & error suite
---
```

&j 2. Core Features Breakdown

- Automatic Local Rooming: Users on the same LAN (e.g. `192.168.1.x`) are automatically grouped into a shared private subnet room (`LAN-192-168-1`).
- Public Router Isolation: Public users are grouped by hashed public IP (`IP-AF2ACBF5`) with privacy masking (`103.21.\*.\*`).
- Zero Sign-Up Required: Instant ephemeral access without emails or phone numbers.

- Custom Room ID & Password: Create rooms with hashed passwords (`SHA-256`).
 - Host Approval ("Knocking"):
 - Hosts can toggle `"Require Host Approval"` when creating or editing custom rooms.
 - When enabled, joining users enter a `"Knocking... Please wait"` overlay state (`ApprovalModal.jsx`).
 - The room host receives an instant glowing knock badge and dialog to Accept or Deny candidate entry.
 - Conditional Leave Room: The `"Leave Room"` button only appears in custom created/joined rooms, keeping users seamlessly connected to their local network by default.
-
- Mobile Voice Notes: Tap-to-record voice audio clips with inline wave duration displays.
 - Voice Toggle: Room hosts can toggle Voice Messaging on or off via Room Settings (`allowVoice`).
 - File Uploads: Drag-and-drop or tap file attachments with preview badges.
-
- Direct P2P Calling: Mesh WebRTC signaling for direct low-latency audio/video calls between room members.
 - Call Controls: Mute mic, toggle camera, flip camera (mobile devices), and end call buttons.
-
- System Notification Cleanup: System join/left announcements auto-purge after 30 minutes to prevent chat clutter.
 - Message TTL Engine: In-memory message TTL purges chat messages older than 30 minutes in `roomManager.js`.

3. Design System & Aesthetic Architecture

Vision features a custom Uiverse (by Praashoo7) Cyber-Neon Dark Glass aesthetic:

```
```.css
/* Color Palette System */
--bg-main: #04060a; /* Deep Space Void */
--card-bg: #05080f; /* Obsidian Panel Glass */
--border-dark: #1a263d; /* Subdued Steel Border */
--neon-green: #00ff88; /* Cyber Matrix Green Accent */
--neon-cyan: #00f0ff; /* Electric WebRTC Cyan Accent */
--warning-amber: #ffb700; /* Key & PIN Amber Highlight */
--error-rose: #f43f5e; /* Knocking Rejection Rose */
```.
```

1. Compact & Sleek Modals:

- `ShareModal.jsx`: Ultra-sleek `max-w-sm` container with a 165px QR canvas and `w-3.5 h-3.5` mini icons.

2. Mobile-First Responsive Layout:

- Utilizes `100dvh` (dynamic viewport height) in `index.css` to prevent mobile browser search bar and virtual keyboard overflows.
- Built-in iOS safe area inset padding (`pt-safe`, `pb-safe`).
- Minimum `44px` touch targets for touch accessibility.

3. Typography: Mono-spaced telemetry font stack (`font-mono`) with crisp contrast hierarchy.

4. Automated Testing & Verification Suite

Vision contains a 28-point automated unit and error test suite ([server/test_full_suite.js](file:///c:/Users/a/OneDrive/Documents/Vision/server/test_full_suite.js)):

Category	Test Case	Target Output	Status
----	----	----	----
IP Subnet	LAN IP Matching (`192.168.1.105`)	`LAN-192-168-1`	`PASSED`
IP Subnet	Public IP Matching (`103.21.244.12`)	`IP-AF2ACBF5`	`PASSED`
IP Subnet	Null IP fallback	`127.0.0.1`	`PASSED`
Room Creation	Custom room creation with host info	`SUITE-ROOM-1`	`PASSED`
Room Validation	Duplicate room creation attempt	`Room ID already exists`	`PASSED`
Password Auth	Valid SHA-256 room password verification	`true`	`PASSED`
Password Auth	Invalid room password rejection	`requiresPassword: true`	`PASSED`
Host Approval	Pending knocking candidate registration	Candidate stored in map	`PASSED`
Host Approval	Host settings update by non-host	`Only the room host can update`	`PASSED`

```
|
|  **Messaging** | Voice note payload & duration validation | `type: audio` | `PASSED` |
|  **Messaging** | 30-Minute message TTL auto-purge | Old messages purged | `PASSED` |
|  **HTTP Server** | `GET /api/health` REST endpoint | `200 OK ({ status: "ok" })` | `PASSED` |
|
|
|
```

5. Deployment & Execution Commands

```
```bash
```

### Start Client Dev Server

---

```
npm run dev --prefix client
```

### Start Backend Server

---

```
node server/index.js
```
```

```
```bash
```

### Build Vite Client Production Bundle

---

```
npm run build --prefix client
```

### Run 28-Point Full Automated Test Suite

---

```
node server/test_full_suite.js
```
```

